A Command-Line Personal Information Manager

Three requirements, the design, and one lesson learned

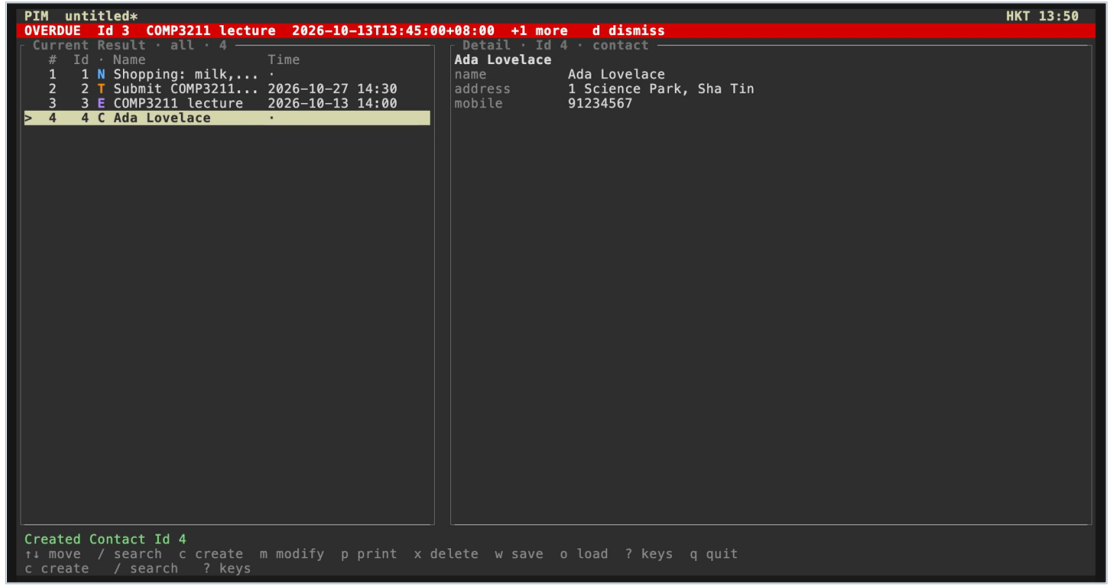
Group [number]

A [Member 1 name]
[Student ID]

B [Member 2 name]
[Student ID]

C [Member 3 name]
[Student ID]

D [Member 4 name]
[Student ID]



The full-screen interface: alarm band, Current Result, and details.

THE SYSTEM

One place for notes, tasks, events, and contacts

N Note

`text` (required)

A quick plain-text note

T Task

`description` (required)

`deadline` (optional)

E Event

`description`, `start` (required)

Any number of alarms, relative or absolute

C Contact

`name` (required)

`address`, `mobile` (optional)

Every record can be

create · **modify** · **search** · **print** · **delete** · **save / load .pim**

Each record keeps a permanent Id · Python 3 standard library only · runs in the macOS Terminal

REQUIREMENT 1 OF 3

Creating a PIR

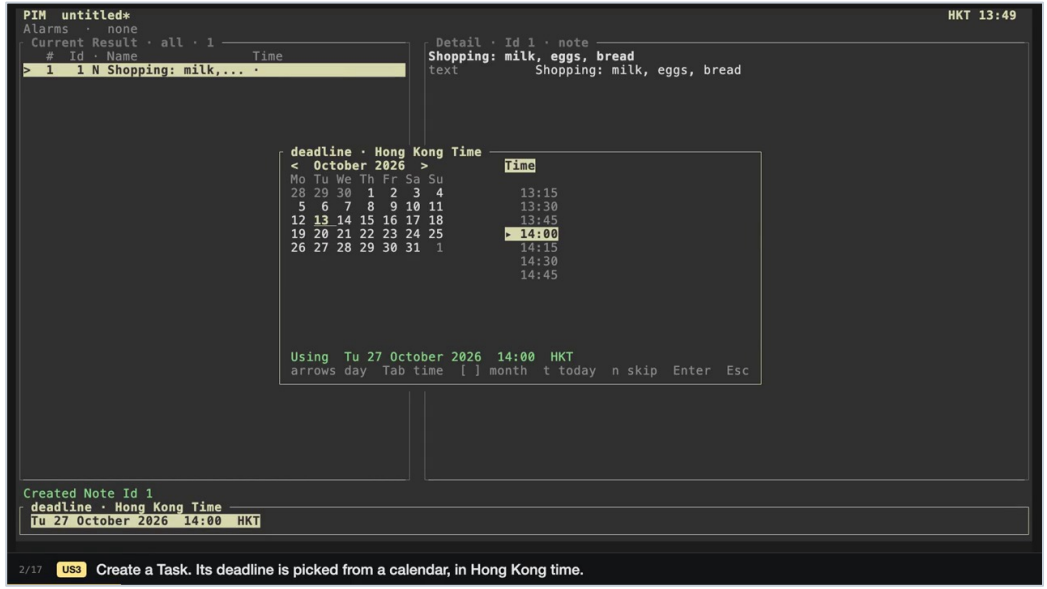
FR-1 `create task` starts a record of one of exactly four types: Note, Task, Event, or Contact.

FR-3 The system asks for one field at a time; pressing Enter skips an optional field.

FR-10 A missing required field cancels the create and names the field: `description is required`.

FR-2 The new record gets the next Id (1, 2, 3 …), and an Id is never reused.

FR-6 An Event can carry any number of alarms, relative to its start or at a fixed time.



Clip (10 s, click to play): creating a Task. The deadline is picked from a calendar in Hong Kong Time.

REQUIREMENT 2 OF 3

Defining a search criterion

FR-16 `search <criterion>` takes one line: tests on type, text, and time, joined by logic.

Type

```
type = event
```

Exactly one of the four types

Text

```
description contains "COMP"
contains "ada"
```

Case-insensitive; without a field name, any text field

Time

```
deadline < 2026-11-20 20:00
alarm = 2026-11-13 13:45
```

On `deadline`, `start`, or `alarm`, to the minute

Precedence

`!` not: binds first → `&&` and: binds next → `||` or: binds last

Parentheses override the order: `type = contact || !contains "comp"` means `type = contact || (!contains "comp")`

REQUIREMENT 3 OF 3

Searching for PIRs

FR-18 `contains` ignores case and matches any part of the text. It is never fuzzy.

FR-20 Times are compared as instants, to the minute. A missing field never matches; `alarm` matches if any one alarm does.

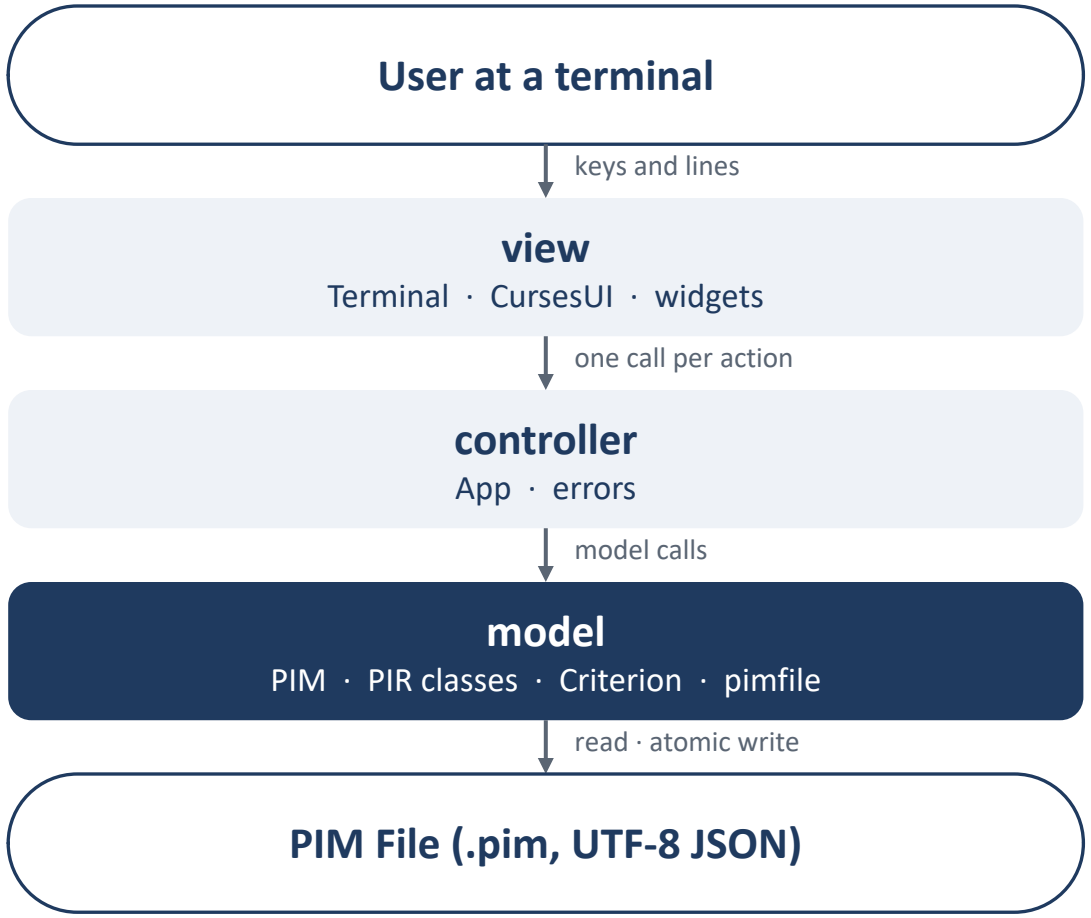
FR-22 The matches replace the list in Id order, and the first one is selected: `2 match(es)`.

FR-23 A syntax error changes nothing, and the typed line stays so it can be corrected.



Clip (13 s, click to play): every contact, plus every record that does not mention "comp".

Model–View–Controller



Why MVC

1. The brief asks for all model code in a separate `model` package.
2. It fits the program's loop: read an action, change the data, redraw.
3. Two front ends, full-screen and line-by-line, share one controller.
4. The model is tested without a screen or a real clock.

How the pattern is instantiated

```
# pim.py, the composition root
def main():
    pim = PIM()           # model
    app = App(pim)        # controller
    Terminal(app).run()   # view
```

Dependencies point one way



`model` imports neither `view` nor `controller`, so its unit tests need no interface at all.

M

`model/`

PIM, the PIR classes, alarms, Criterion, pimfile. Every rule and the file format.

C

`controller/`

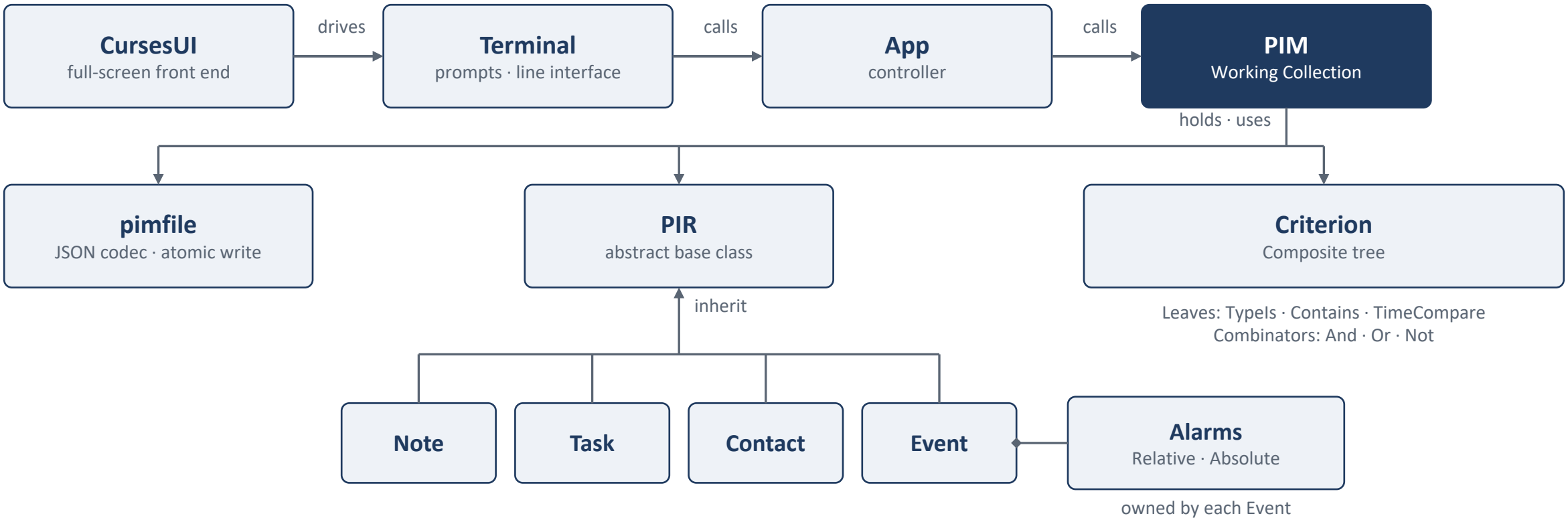
App and errors. One completed user action becomes model calls and one status line.

V

`view/`

Terminal, CursesUI, widgets. The event loop, prompts, Alarm Alerts, and drawing.

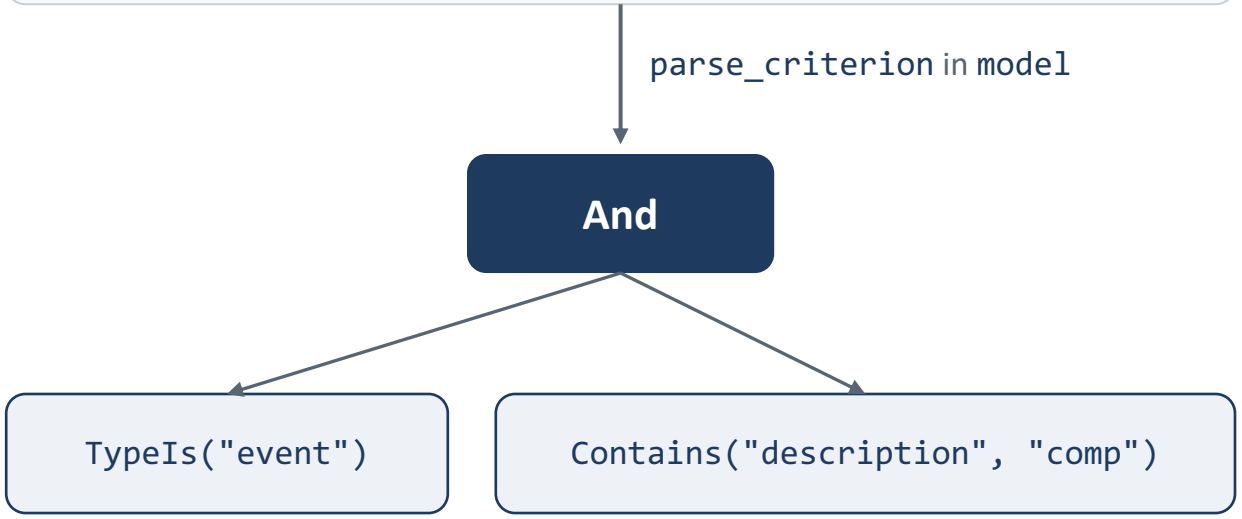
The main classes and how they relate



Arrows show calls and use; the bar under PIR joins its four subclasses; the diamond marks ownership. The design document lists every field and method.

A criterion is a tree built inside the model

```
search type = event && description contains "COMP"
```



`PIM.search` asks the root `matches(pir)` for every record; `And` asks its children.

1. **Composite pattern.** Leaves test one record; `And`, `Or`, and `Not` combine smaller criteria.
2. **The parser lives in the model.** `parse_criterion` turns one line into the tree, with `!` before `&&` before `||`.
3. **So all of US7 can be unit-tested**, grammar and matching, without the view.

Put every rule where a unit test can reach it

Problem

Search mixes field tests, three time operators, missing values, alarm lists, and operator precedence.

Alarms also depend on the clock. Testing all that through the terminal needs scripted keys and real time.

Decision

We decided early to keep every rule in `model`.

`parse_criterion` is a plain function, and `due_alarms(now)` takes the time as an argument.

The search tests share one six-PIR fixture.

Result

100 %

line coverage of `model`: 800 of 800 lines

Search is tested with plain `unittest` calls: no screen, no clock.

281 unit tests run in well under a second.

Takeaway

If a rule cannot be tested through the model's interface, it is in the wrong place.

SUMMARY

What we built and what we learned

11 / 11
user stories implemented

53 / 53
SRS requirements implemented

341
automated tests, all passing

100 %
line coverage of model

1 Requirements

Create a PIR, define a criterion, and search, each stated so it can be tested.

2 Design

MVC in three packages, with the model as the test surface and criteria as a Composite tree.

3 Lesson

Testability decided where code lives: every rule sits where a unit test can reach it.

Thank you.